

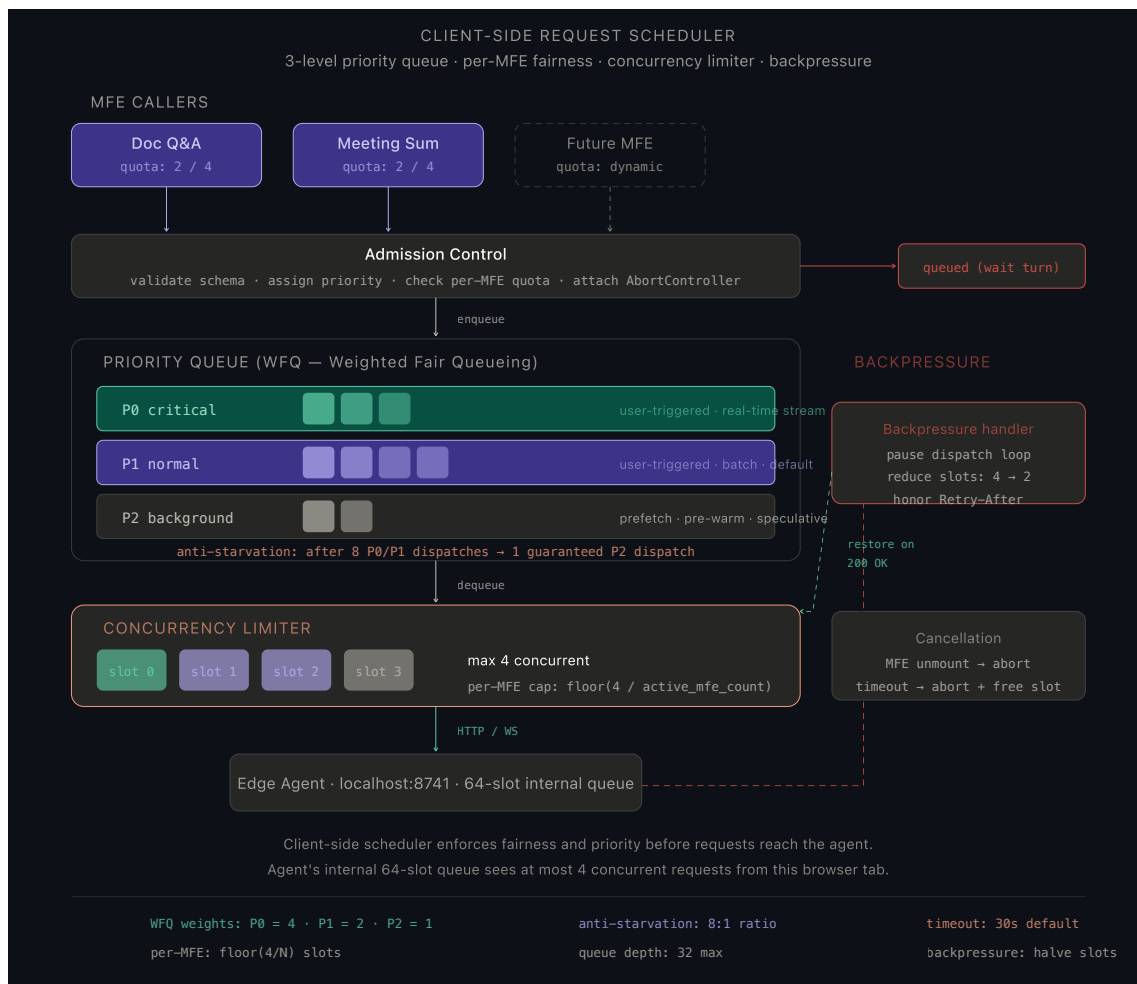
Part B §2 — Client-Side Request Scheduler

Constraint: The Edge Agent can handle a maximum of **4 concurrent inference requests**. Additional requests must queue. The scheduler must enforce per-MFE fairness, support priority-based scheduling, handle cancellation and timeout, and prevent starvation of low-priority workloads.

Overview

The scheduler is not a standalone service — it is a component inside the shell's `EdgeAgentService` (described in Part B §1). It sits between the MFEs' `inference()` / `stream()` calls and the HTTP/WebSocket connection to the Edge Agent.

Its job: accept requests from N MFEs, enforce a **4-slot concurrency limit**, apply **per-MFE fairness** and **priority ordering**, and handle **cancellation** and **timeout** without leaking slots or starving any workload.



Assumptions

1. **4-slot concurrency limit** is the agent's hard maximum. Sending a 5th concurrent request results in `429 Too Many Requests`. The scheduler must never exceed this.

- 2. **Two MFEs currently active:** Document Q&A (batch translation) and Meeting Summariser (real-time streaming). The scheduler must scale to N MFEs without code changes.
- 3. **Three priority levels are sufficient:** critical (user-triggered real-time), normal (user-triggered batch), and background (prefetch/speculative). This covers all current and foreseeable MFE workloads.
- 4. **Cancellation must propagate to the agent.** Aborting a request client-side must close the HTTP connection so the agent can free its internal queue slot.
- 5. **Requests are independent** — no ordering dependencies between requests from the same or different MFEs. The scheduler does not need to preserve FIFO order across MFEs.

1. Queue Structure

Three-level priority queue with Weighted Fair Queueing

The scheduler uses three priority lanes, each a FIFO queue. The lanes are served using **Weighted Fair Queueing (WFQ)** — not strict priority — to prevent starvation.



Priority levels

Level	Name	Weight	Use cases	Timeout
P0	critical	4	User-triggered real-time: live transcription, interactive TTS, streaming Q&A	30s

P1	normal	2	User-triggered batch: document translation, summarisation, search	30s
P2	background	1	Prefetch, speculative pre-computation, model warm-up	60s

How MFEs assign priority

Priority is set by the MFE at call time via the `priority` field:

```
// Real-time transcription – critical
await stream({ model: 'indic-asr-v1', input: audio, priority: 'critical', caller:
'meeting-sum' });

// User clicks "Translate" – normal
await inference({ model: 'indic-nmt-v2', input: text, priority: 'normal', caller:
'doc-qa' });

// Pre-fetch next page's translation – background
await inference({ model: 'indic-nmt-v2', input: nextPageText, priority:
'background', caller: 'doc-qa' });
```

The scheduler validates priority but does not override it. MFEs are trusted to classify their own requests correctly — they know their UX context better than the scheduler.

Queue entry

Each queued request is represented as:

```
interface QueueEntry {
  id: string; // unique request ID
  caller: string; // MFE identifier ('doc-qa', 'meeting-sum')
  priority: 'critical' | 'normal' | 'background';
  request: InferenceRequest; // model, input, params
  enqueuedAt: number; // Date.now() – for starvation detection
  abortController: AbortController;
  resolve: (result: InferenceResult) => void;
  reject: (error: Error) => void;
  timeoutId: number; // setTimeout handle for timeout propagation
}
```

2. Scheduling Policy

Weighted Fair Queueing (WFQ)

Strict priority scheduling (always serve P0 before P1 before P2) causes **starvation** — a continuous stream of P0 real-time transcription requests would permanently block all P1 and P2 work. WFQ prevents this.

Each priority lane has a **virtual time** counter. When a request is enqueued, its virtual finish time is computed as:

```
virtual_finish_time = virtual_time + (1 / weight)
```

The dequeue function picks the non-empty lane with the **lowest virtual finish time**. Because P0 has weight 4 and P2 has weight 1, P0 gets dequeued 4× as often as P2 — but P2 still gets served.

```
function dequeue(): QueueEntry | null {
  // Anti-starvation hard guard: after 8 consecutive P0/P1 dispatches, force a P2
  if (consecutiveHighPriority >= 8 && queues.background.length > 0) {
    consecutiveHighPriority = 0;
    return queues.background.dequeue();
  }

  // WFQ: pick lane with lowest virtual finish time
  let bestLane: PriorityLane | null = null;
  let bestTime = Infinity;

  for (const lane of [queues.critical, queues.normal, queues.background]) {
    if (lane.length > 0 && lane.virtualFinishTime < bestTime) {
      bestTime = lane.virtualFinishTime;
      bestLane = lane;
    }
  }

  if (!bestLane) return null;

  const entry = bestLane.dequeue();
  bestLane.virtualFinishTime += 1 / bestLane.weight;

  if (entry.priority !== 'background') consecutiveHighPriority++;
  else consecutiveHighPriority = 0;

  return entry;
}
```

Dispatch ratio under load

With all three lanes full and continuous arrivals:

Lane	Weight	Share of dispatches	Example: 7 dispatches
P0 (critical)	4	4/7 = 57%	4 dispatches
P1 (normal)	2	2/7 = 29%	2 dispatches
P2 (background)	1	1/7 = 14%	1 dispatch

Plus the hard anti-starvation guard: after 8 consecutive P0/P1 dispatches without any P2, a P2 request is forced to the front. This guarantees P2 is served at least once every 9 dispatches regardless of WFQ timing.

Priority preemption

When a P0 `critical` request arrives and all 4 concurrency slots are occupied by P2 `background` requests, the scheduler **preempts**:

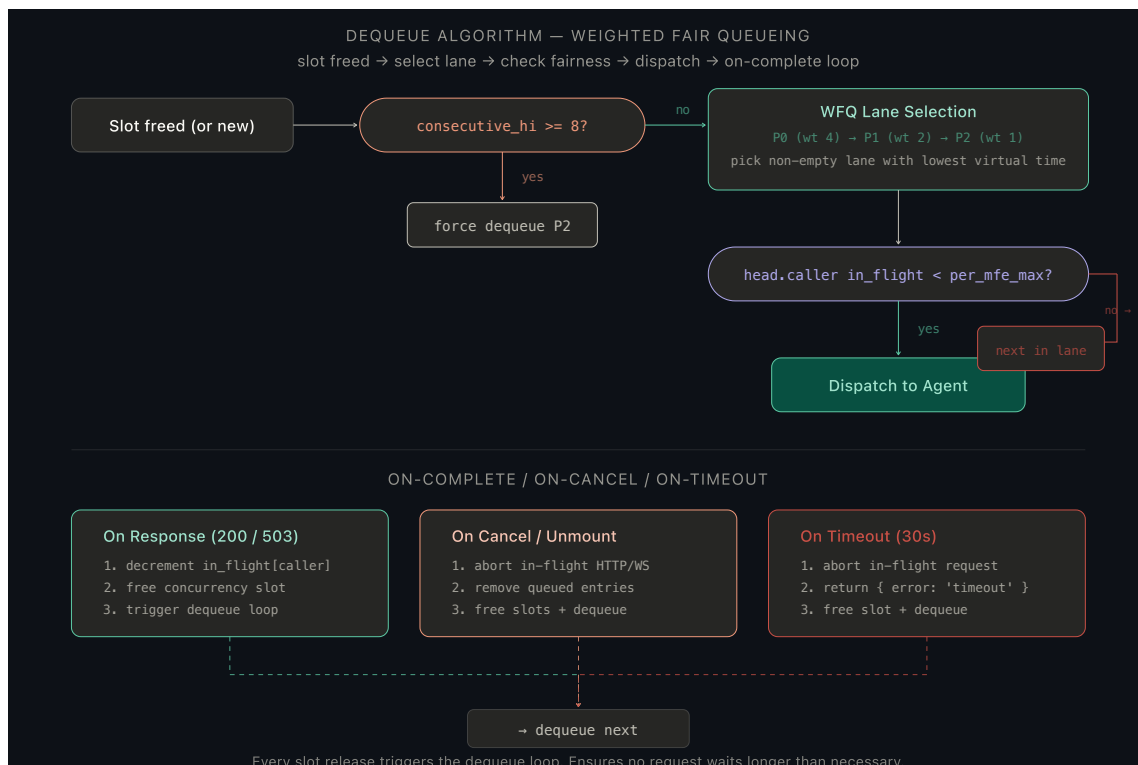
1. Selects the youngest (most recently dispatched) P2 request
2. Aborts its HTTP connection via `AbortController.abort()`
3. Re-enqueues the aborted P2 request at the head of the P2 lane (so it's retried next)
4. Dispatches the P0 request into the freed slot

Preemption only applies to P0 preempting P2. P0 does not preempt P1, and P1 does not preempt P2. This keeps the policy simple and predictable.

```
function maybePreempt(entry: QueueEntry): boolean {
  if (entry.priority !== 'critical') return false;
  if (inFlightCount < maxConcurrent) return false; // slot available, no
  // preemption needed

  // Find a background request to preempt
  const victim = findYoungestInFlight('background');
  if (!victim) return false; // no background requests to preempt

  victim.abortController.abort();
  queues.background.prepend(victim); // re-queue at head for retry
  freeSlot(victim);
  return true;
}
```



3. Concurrency Management

Per-MFE fairness

The concurrency limiter enforces two constraints simultaneously:

1. **Global limit:** at most 4 requests in-flight total
2. **Per-MFE limit:** at most `floor(4 / active_mfe_count)` requests in-flight per MFE

With 2 active MFEs, each gets at most 2 concurrent slots. With 3 MFEs, each gets 1 slot (with 1 slot available for whoever's next in the queue). With 1 MFE, it gets all 4.

```
function canDispatch(entry: QueueEntry): boolean {
  if (totalInFlight >= maxConcurrent) return false;

  const activeMFEs = countDistinctCallers();
  const perMFEMax = Math.max(1, Math.floor(maxConcurrent / activeMFEs));

  if (callerInFlight[entry.caller] >= perMFEMax) return false;

  return true;
}
```

Dynamic quota adjustment

The per-MFE quota is **recalculated on every dispatch decision**, not fixed at configuration time. This means:

- When Meeting Summariser unmounts (1 MFE remaining), Document Q&A's quota rises from 2 to 4
- When a new MFE loads, all quotas shrink proportionally
- No manual configuration needed — the formula adapts to N MFEs

Slot accounting

```
class ConcurrencyLimiter {
  private maxConcurrent = 4;
  private totalInFlight = 0;
  private callerInFlight: Map<string, number> = new Map();

  acquire(caller: string): boolean {
    if (!this.canDispatch(caller)) return false;
    this.totalInFlight++;
    this.callerInFlight.set(caller, (this.callerInFlight.get(caller) ?? 0) + 1);
    return true;
  }

  release(caller: string): void {
    this.totalInFlight--;
    const current = this.callerInFlight.get(caller) ?? 1;
    if (current <= 1) this.callerInFlight.delete(caller);
    else this.callerInFlight.set(caller, current - 1);
  }
}
```

```
        // Trigger dequeue loop – a slot is free
        this.onSlotFreed();
    }
}
```

The dispatch loop

The scheduler uses an **event-driven dispatch loop**, not polling. The loop runs whenever:

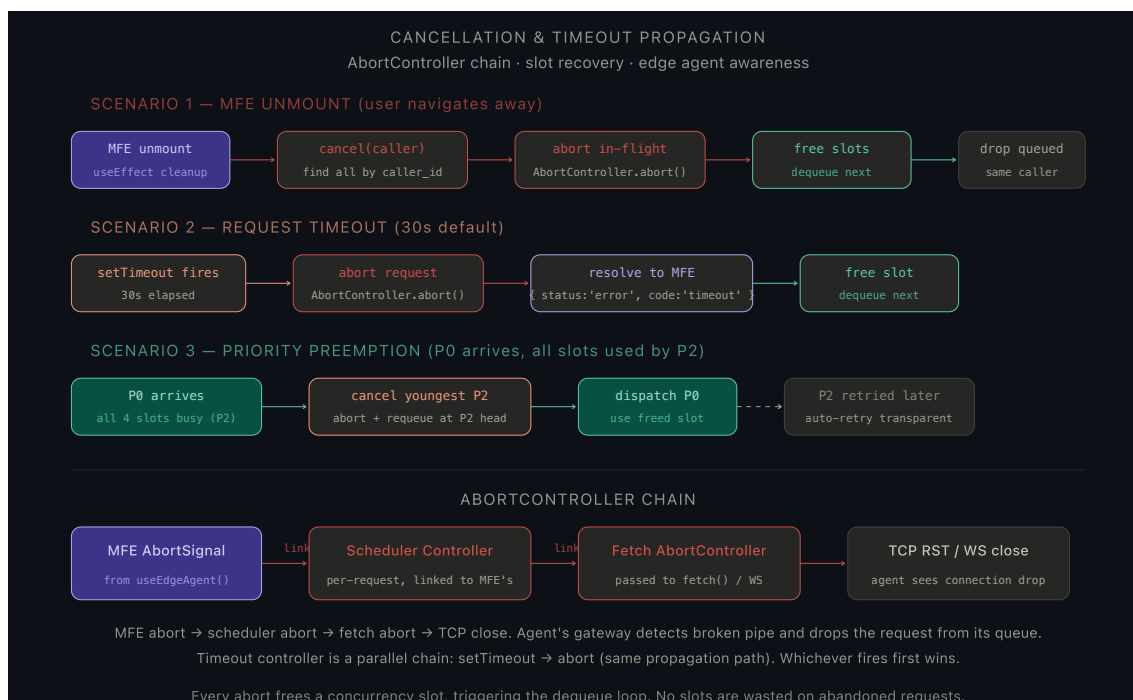
- A new request is enqueued
- A slot is freed (response, cancellation, or timeout)
- Backpressure is lifted (after a 429 recovery)

```
function dispatchLoop(): void {
    while (totalInFlight < maxConcurrent) {
        const entry = dequeue();
        if (!entry) break; // all queues empty

        if (!canDispatch(entry)) {
            // This MFE hit its per-MFE quota – skip to next in queue
            requeue(entry);
            continue;
        }

        acquire(entry.caller);
        dispatch(entry); // sends HTTP/WS request to agent
    }
}
```

4. Cancellation and Timeout Propagation



AbortController chain

Every request has a three-level abort chain:

```
MFE's AbortSignal → Scheduler's AbortController → fetch() AbortController
(from hook)          (per-request)          (HTTP/WS)
```

When any level aborts:

1. The signal propagates down the chain
2. The HTTP connection is closed (TCP RST or WebSocket close frame)
3. The Edge Agent detects the broken pipe and drops the request from its internal queue
4. The concurrency slot is freed
5. The dequeue loop fires, dispatching the next waiting request

MFE unmount cancellation

When an MFE unmounts, its React cleanup function calls `cancel(caller)` :

```
// Inside EdgeAgentProvider
useEffect(() => {
  return () => {
    // MFE is unmounting – cancel all its pending work
    scheduler.cancelByCaller(callerRef.current);
  };
}, []);
```

The scheduler finds all entries (queued and in-flight) with that `caller` ID and aborts them:


```

cancelByCaller(caller: string): void {
  // 1. Abort in-flight requests
  for (const entry of this.inFlight.values()) {
    if (entry.caller === caller) {
      entry.abortController.abort();
      clearTimeout(entry.timeoutId);
      this.release(caller);
    }
  }

  // 2. Remove queued requests
  for (const lane of [this.queues.critical, this.queues.normal,
this.queues.background]) {
    lane.removeByCaller(caller);
  }
}

```

Timeout propagation

Every request gets a timeout timer at enqueue time:

```

function enqueue(entry: QueueEntry): void {
  const timeoutMs = entry.priority === 'background' ? 60_000 : 30_000;

  entry.timeoutId = setTimeout(() => {
    entry.abortController.abort();
    entry.resolve({
      status: 'error',
      code: 'timeout',
      message: `Request timed out after ${timeoutMs / 1000}s`,
      retrying: false,
    });
    this.release(entry.caller);
  }, timeoutMs);

  // ... add to priority lane
}

```

The timeout covers both **queue wait time** and **in-flight time**. A request that waits 25s in the queue and then takes 6s for inference will timeout at 30s total. This prevents slow-queue scenarios from silently accumulating latency.

Timeout by priority

Priority	Timeout	Rationale
critical	30s	User is waiting for real-time output. 30s is the UX upper bound.
normal	30s	User triggered this. Same UX bound.

background	60s	No user waiting. Allow longer queue time. Still bounded to prevent resource leaks.
------------	-----	--

5. Starvation Prevention

The starvation problem

Without protection, low-priority workloads can be permanently blocked:

- Meeting Summariser continuously streams P0 `critical` ASR requests
- Document Q&A submits P2 `background` prefetch requests
- Under WFQ alone, P2 gets 1/7 of dispatches — but if P0 arrivals outpace P2 dispatches, the P2 queue grows unbounded and eventually times out

Three starvation defenses

Defense 1: WFQ weights guarantee minimum share

WFQ mathematically guarantees each lane gets at least `weight / total_weight` of dispatches. P2 gets at least 1/7 (14%) of all dispatches as long as slots are available.

Defense 2: Hard anti-starvation guard (8:1 ratio)

After 8 consecutive dispatches from P0/P1 lanes without any P2 dispatch, the scheduler **forces** a P2 dequeue regardless of virtual finish times. This is the safety net — it guarantees P2 gets at least 1 dispatch per 9 cycles.

Defense 3: Age-based promotion

If a P2 request has been queued for more than 10 seconds, it is **promoted** to P1. If a P1 request has been queued for more than 20 seconds, it is promoted to P0. This prevents any request from waiting indefinitely.

```
function maybePromote(): void {
  const now = Date.now();

  for (const entry of queues.background) {
    if (now - entry.enqueueAt > 10_000) {
      queues.background.remove(entry);
      entry.priority = 'normal';
      queues.normal.enqueue(entry);
    }
  }

  for (const entry of queues.normal) {
    if (now - entry.enqueueAt > 20_000) {
      queues.normal.remove(entry);
      entry.priority = 'critical';
      queues.critical.enqueue(entry);
    }
  }
}
```

Promotion is checked on every dequeue cycle. It adds ~0.1ms overhead (iterating at most 32 entries).

Starvation guarantees

Workload	Guarantee	Mechanism
P2 background	Served at least once every 9 dispatches	WFQ + 8:1 hard guard
P2 background	Promoted to P1 after 10s wait	Age-based promotion
P1 normal	Gets 2/7 of dispatches under contention	WFQ weight = 2
P1 normal	Promoted to P0 after 20s wait	Age-based promotion
Any request	Times out after 30s (60s for P2)	Timeout propagation

6. Backpressure Handling

When the agent pushes back

Even with the 4-slot limit, the agent can return `429 Too Many Requests` if other clients (Electron apps, other browser tabs, extensions) are also sending requests.

Backpressure protocol

```
Agent returns 429 + Retry-After: 2
|
├─ 1. Pause dispatch loop immediately
|   → No new requests sent to agent
|
├─ 2. Reduce maxConcurrent: 4 → 2
|   → Halve the concurrency limit
|   → Don't abort in-flight – let them complete
|
├─ 3. Start Retry-After timer (2s)
|   → On timer: resume dispatch loop at reduced concurrency
|
├─ 4. On next successful 200 OK:
|   → Increment maxConcurrent back toward 4 (additive increase)
|   → +1 per successful response, capped at 4
|
└─ 5. MFE experience:
    → Longer queue times, not errors
    → No 429 exposed to MFEs
```

AIMD — Additive Increase, Multiplicative Decrease

The concurrency limit follows AIMD (the same algorithm as TCP congestion control):

- On `429` : multiply by 0.5 (4 → 2, 2 → 1)
- On `200` : add 1 (1 → 2 → 3 → 4)

This converges to the agent's actual available capacity without oscillation.

```
function onResponse(status: number, retryAfter?: number): void {
  if (status === 429) {
    // Multiplicative decrease
    maxConcurrent = Math.max(1, Math.floor(maxConcurrent / 2));
    pauseDispatchFor(retryAfter * 1000);
  } else if (status === 200) {
    // Additive increase
    if (maxConcurrent < 4) {
      maxConcurrent++;
    }
  }
}
```

What MFEs see during backpressure

MFEs never see `429`. The scheduler absorbs backpressure by:

1. Queuing requests locally (up to 32 entries)
2. Slowing down dispatch rate
3. Returning results as they complete (just slower)

If the queue fills to 32 entries, new requests are rejected with `{ status: 'error', code: 'queue_full' }`. This is the last-resort backpressure signal to MFEs — they should show a "system busy" UI.

7. Process Lifecycle

The scheduler has six internal states. MFEs do not observe these states directly — they are exposed indirectly through `queued_ms`, `connected`, and error codes returned by `inference()` / `stream()`.

 Scheduler Lifecycle State Machine

States

State	Entry condition	Queue behavior	Slot behavior	MFE experience
IDLE	No requests pending or in-flight	Accept + dispatch immediately	0/4 used	Instant results
DISPATCHING	At least 1 request in-flight, slots available	WFQ dequeue active	1–3/4 used	May see brief <code>queued_ms</code>
SATURATED	All 4 slots occupied, queue non-empty	Enqueue; P0 may preempt P2	4/4 used	Loading spinner
BACKPRESSURE	429 received from agent	Enqueue; dispatch paused	AIMD: max reduced to 1–2	Longer wait times

SUSPENDED	Agent disconnected (ECONNREFUSED / WebSocket close)	Enqueue with 30s timeout	All frozen, no dispatch	"AI offline" banner
DRAINING	Agent reconnected after SUSPENDED	Rapid WFQ dequeue	Filling 0 → 4	Burst of results

Transitions

```

IDLE —request arrives→ DISPATCHING
DISPATCHING —4/4 full→ SATURATED
SATURATED —slot freed→ DISPATCHING
DISPATCHING —queue empty + all done→ IDLE
DISPATCHING —429 received→ BACKPRESSURE
SATURATED —429 received→ BACKPRESSURE
BACKPRESSURE —Retry-After + 200 OK→ DISPATCHING
DISPATCHING —ECONNREFUSED→ SUSPENDED
SATURATED —ECONNREFUSED→ SUSPENDED
SUSPENDED —connection:restored→ DRAINING
DRAINING —queue flushed→ DISPATCHING

```

Implementation

```

type SchedulerState = 'idle' | 'dispatching' | 'saturated' | 'backpressure' |
'suspended' | 'draining';

class SchedulerStateMachine {
  private state: SchedulerState = 'idle';

  transition(event: SchedulerEvent): void {
    const prev = this.state;
    switch (this.state) {
      case 'idle':
        if (event === 'request_arrived') this.state = 'dispatching';
        break;
      case 'dispatching':
        if (event === 'slots_full') this.state = 'saturated';
        else if (event === 'all_done') this.state = 'idle';
        else if (event === '429_received') this.state = 'backpressure';
        else if (event === 'agent_disconnected') this.state = 'suspended';
        break;
      case 'saturated':
        if (event === 'slot_freed') this.state = 'dispatching';
        else if (event === '429_received') this.state = 'backpressure';
        else if (event === 'agent_disconnected') this.state = 'suspended';
        break;
      case 'backpressure':
        if (event === 'retry_ok') this.state = 'dispatching';
        break;
      case 'suspended':

```

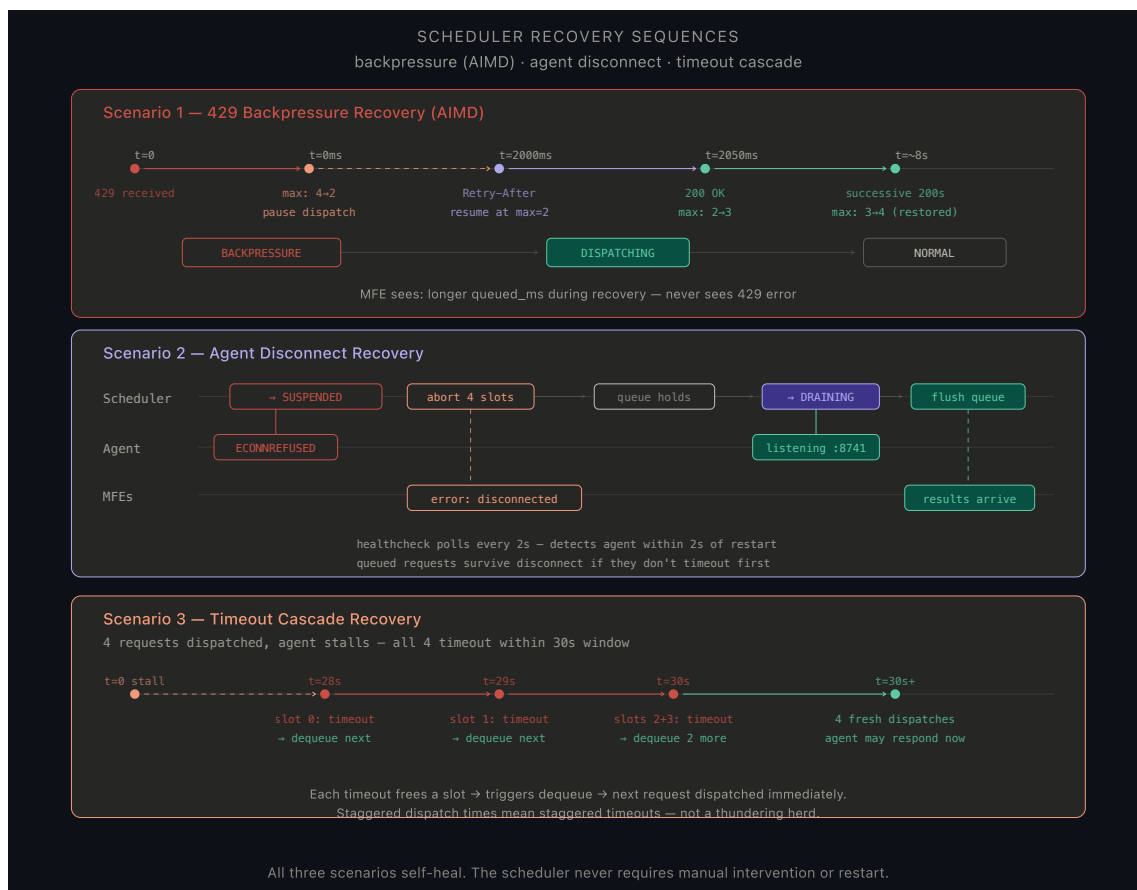
```

        if (event === 'connection_restored') this.state = 'draining';
        break;
      case 'draining':
        if (event === 'queue_flushed') this.state = 'dispatching';
        break;
      }
      if (prev !== this.state) {
        this.emit('scheduler:state_change', { from: prev, to: this.state });
      }
    }
  }
}

```

8. Recovery Sequences

The scheduler self-heals from three failure scenarios without manual intervention or restart.



Scenario 1 — 429 Backpressure Recovery (AIMD)

Time	Event	Scheduler action	State
$t=0$	Agent returns 429 Retry-After: 2	Halve maxConcurrent (4→2), pause dispatch	BACKPRESSURE

t=2s	Retry-After timer expires	Resume dispatch at reduced concurrency (max=2)	DISPATCHING
t=~2.05s	Next response is 200 OK	Additive increase: max 2→3	DISPATCHING
t=~4s	Another 200 OK	Additive increase: max 3→4	DISPATCHING
t=~8s	Steady state restored	Full concurrency, no queueing	DISPATCHING or IDLE

MFE experience: longer `queued_ms` during recovery window. No error codes exposed.

Scenario 2 — Agent Disconnect Recovery

Time	Event	Scheduler action	State
t=0	ECONNREFUSED or WebSocket close	Abort all 4 in-flight requests, return { error: 'disconnected' }	SUSPENDED
t=0+	New requests arrive	Enqueued with 30s timeout (not rejected)	SUSPENDED
t=0–30s	Health-check polls every 2s	Probing localhost:8741 for reconnection	SUSPENDED
t=Xs	Agent starts listening again	Health-check succeeds, connection:restored event	DRAINING
t=Xs+	Queued requests dispatched	Rapid WFQ dequeue, slots fill 0→4	DRAINING → DISPATCHING

If the agent is down longer than 30s, queued requests timeout individually. New requests submitted during SUSPENDED still get a 30s window — the timeout starts at enqueue time, not at the disconnect event.

Scenario 3 — Timeout Cascade Recovery

When the agent stalls (accepting connections but not responding), all 4 in-flight requests will timeout at staggered intervals:

Time	Event	Result
t=28s	Oldest request (dispatched at t=0) hits 30s total timeout	Abort, free slot, dequeue next
t=29s	Second-oldest times out	Free slot, dequeue next
t=30s	Remaining 2 requests timeout	Free 2 slots, dequeue 2
t=30s+	4 fresh requests dispatched	If agent recovered, results arrive normally

Key design: because requests are dispatched at different times (staggered by slot availability), their timeouts fire at different times. This prevents a thundering herd where all 4 slots free simultaneously and 4 new requests hit a still-stalled agent.

Sweep timer safety net

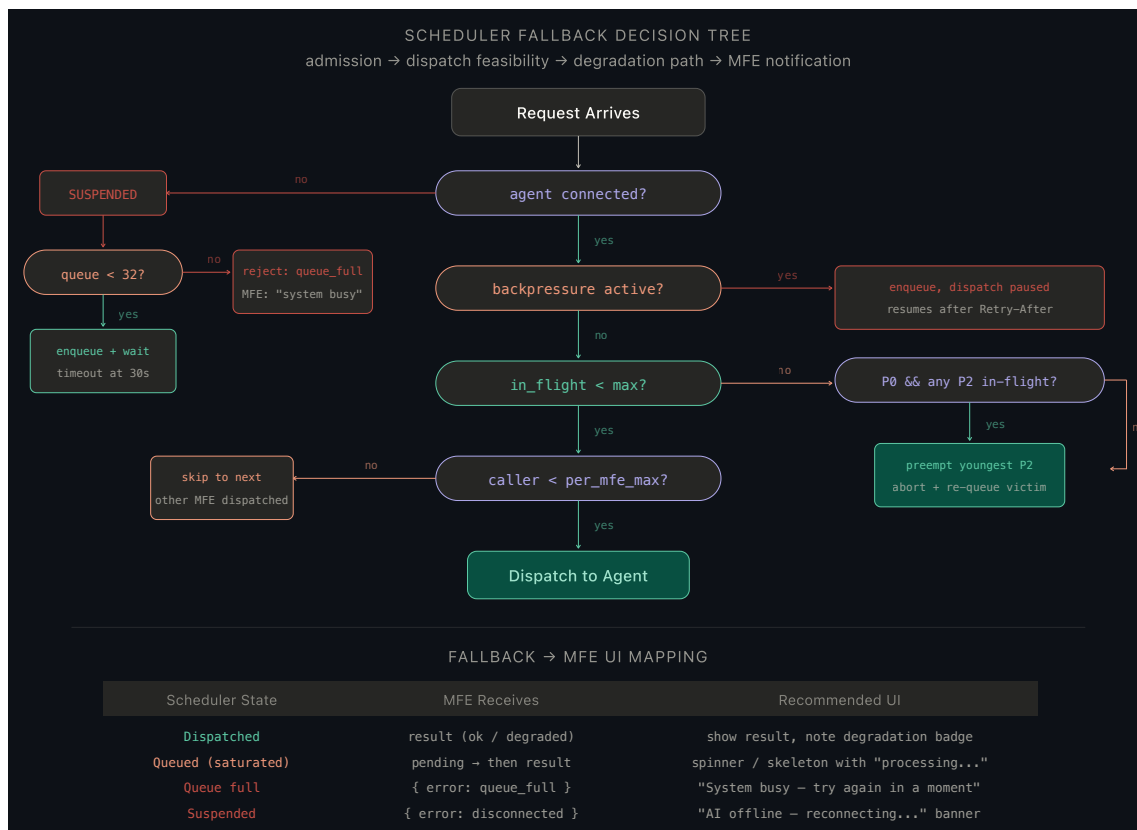
As a last resort against slot leaks, a 60-second sweep timer runs periodically:

```
setInterval(() => {
  const now = Date.now();
  for (const [id, entry] of inFlight.entries()) {
    if (now - entry.dispatchedAt > 60_000) {
      entry.abortController.abort();
      release(entry.caller);
      entry.resolve({
        status: 'error',
        code: 'slot_leak_recovery',
        message: 'Request exceeded maximum lifetime (60s)',
        retrying: false,
      });
    }
  }
}, 60_000);
```

This catches edge cases where the normal timeout or abort path fails to fire (e.g., due to a browser timer throttling in background tabs).

9. Fallback Decision Tree

When a request cannot follow the normal dispatch path, the scheduler walks a decision tree to determine the appropriate fallback.



Decision flow

```

Request arrives
├─ Agent disconnected?
│   ├── YES → SUSPENDED
│   │   ├── Queue < 32? → enqueue (timeout 30s, wait for reconnect)
│   │   └─ Queue full? → reject: { error: 'queue_full' }
│   └─ NO ↓
├─ Backpressure active?
│   ├── YES → enqueue, dispatch paused (resumes after Retry-After)
│   └─ NO ↓
├─ Slot available (in_flight < max)?
│   ├── NO → is P0 and any P2 in-flight?
│   │   ├── YES → preempt youngest P2, dispatch P0
│   │   └─ NO → enqueue in priority lane (SATURATED)
│   └─ YES ↓
├─ Per-MFE quota available (caller < per_mfe_max)?
│   ├── NO → skip, dispatch next MFE's request from queue
│   └─ YES ↓
└─ DISPATCH to agent
  
```

Fallback → MFE UI mapping

Each fallback path produces a specific response that MFEs should map to a UI state:


```
P0 [critical]   weight 4   - real-time, user-facing
P1 [normal]    weight 2   - user-triggered batch
P2 [background] weight 1   - prefetch, speculative
```

```
Anti-starvation: 8:1 hard guard + age-based promotion
```



Concurrency Limiter

```
Global max: 4 (AIMD-adjusted: may be 1-4)
Per-MFE max: floor(4 / active_mfe_count)
```

```
If MFE at quota: skip to next entry in queue
If P0 and all slots P2: preempt youngest P2
```



Dispatch

```
Send HTTP POST / WS frame to localhost:8741
Attach AbortController signal to fetch()
Start timeout timer (30s / 60s)
```

```
On response: free slot → dequeue loop
On 429: AIMD decrease → pause → retry
On timeout: abort → free slot → return error to MFE
On MFE unmount: abort → free slot → drop queued
```

11. API Contract Examples

MFE → Scheduler (via `useEdgeAgent()`)

```
// Critical: live transcription
const result = await stream({
  model: 'indic-asr-v1',
  input: audioChunk,
  priority: 'critical',
  caller: 'meeting-sum',
});

// Normal: user-triggered translation
const result = await inference({
  model: 'indic-nmt-v2',
  input: 'Hello, how are you?',
  priority: 'normal',
  caller: 'doc-qa',
});

// Background: prefetch
const result = await inference({
```

```
    model: 'indic-nmt-v2',
    input: nextPageText,
    priority: 'background',
    caller: 'doc-qa',
  });
```

Scheduler → MFE (response types)

```
// Success (immediate dispatch)
{ status: 'ok', data: { text: '...' }, meta: { compute: 'ane', tier: 0, latency_ms:
42, queued_ms: 0 } }

// Success (was queued first)
{ status: 'ok', data: { text: '...' }, meta: { compute: 'ane', tier: 0, latency_ms:
42, queued_ms: 1200 } }

// Degraded (dispatch succeeded but agent is on CPU fallback)
{ status: 'degraded', data: { text: '...' }, meta: { compute: 'cpu-fallback', tier:
2, latency_ms: 185, queued_ms: 0 },
  degradation: { reason: 'ane_faulted', latency_factor: 4.4 } }

// Error: timeout
{ status: 'error', code: 'timeout', message: 'Request timed out after 30s',
retrying: false }

// Error: queue full
{ status: 'error', code: 'queue_full', message: 'Scheduler queue is full (32/32).
Try again later.', retrying: false }

// Error: cancelled (MFE won't see this – it unmounted)
// (The promise is never resolved – it's garbage collected with the MFE)
```

Scheduler → Edge Agent (HTTP — internal)

```
POST /v1/inference HTTP/1.1
Host: 127.0.0.1:8741
Authorization: Bearer ek_live_...
Content-Type: application/json
X-Request-Id: req_7f3a9b
X-Caller-Id: doc-qa
X-Priority: normal

{"model": "indic-nmt-v2", "input": "Hello, how are you?", "params": {"source_lang":
"en", "target_lang": "hi"}}
```

12. State/Event Payload Examples

Scheduler internal state

```
{
  "concurrency": {
    "max": 4,
    "effective_max": 4,
    "in_flight": 3,
    "per_caller": {
      "doc-qa": { "in_flight": 1, "queued": 2, "max": 2 },
      "meeting-sum": { "in_flight": 2, "queued": 0, "max": 2 }
    }
  },
  "queues": {
    "critical": { "length": 0, "virtual_time": 12.25 },
    "normal": { "length": 2, "virtual_time": 11.5 },
    "background": { "length": 1, "virtual_time": 10.0 }
  },
  "starvation": {
    "consecutive_high_priority": 3,
    "oldest_queued_age_ms": 2400,
    "promotions_total": 0
  },
  "backpressure": {
    "active": false,
    "last_429_at": null,
    "aimd_state": "normal"
  }
}
```

Event payloads

```
// Emitted when a request is queued (not immediately dispatched)
{ type: 'scheduler:queued', request_id: 'req_7f3a', caller: 'doc-qa', priority:
'normal', position: 3 }

// Emitted when a request is dispatched to the agent
{ type: 'scheduler:dispatched', request_id: 'req_7f3a', caller: 'doc-qa', queued_ms:
1200 }

// Emitted when backpressure kicks in
{ type: 'scheduler:backpressure', reason: '429_received', max_reduced_to: 2,
retry_after_ms: 2000 }

// Emitted when backpressure lifts
{ type: 'scheduler:backpressure-lifted', max_restored_to: 4 }

// Emitted when a background request is promoted
{ type: 'scheduler:promoted', request_id: 'req_9c1b', from: 'background', to:
'normal', wait_ms: 10200 }

// Emitted when a P2 request is preempted by P0
```

```
{ type: 'scheduler:preempted', victim_id: 'req_2d4e', preemptor_id: 'req_8a1f',  
victim_requeued: true }
```

13. Design Tradeoff Explanations

Decision	Chosen	Alternative	Rationale
WFQ over strict priority	Weighted Fair Queueing (4:2:1)	Strict $P0 > P1 > P2$	Strict priority causes starvation. WFQ guarantees every lane gets at least weight/total share while still strongly favoring P0.
3 priority levels	critical / normal / background	2 levels (high/low) or 5+ levels	3 maps cleanly to real-time, interactive, and speculative. 2 is too coarse (can't distinguish prefetch from user click). 5+ adds configuration burden with no UX benefit.
Per-MFE quota = floor(4/N)	Dynamic quota based on active MFE count	Fixed quota per MFE	Dynamic adapts to MFE load/unload without config changes. Fixed quotas waste slots when MFEs are inactive.
P0 preempts P2 only	Critical can abort background	Any higher priority preempts any lower	Limiting preemption to $P0 \rightarrow P2$ keeps the policy simple. P0 preempting P1 would cause too much wasted work (P1 requests are user-triggered).
Age-based promotion	Promote after 10s ($P2 \rightarrow P1$) or 20s ($P1 \rightarrow P0$)	No promotion	Prevents edge cases where WFQ math + queue depth conspire to delay a request beyond its timeout. Simple to implement, adds ~0.1ms overhead.
8:1 hard starvation guard	Force P2 after 8 consecutive P0/P1	Rely on WFQ alone	WFQ is probabilistic under bursty arrivals. The hard guard is a deterministic safety net. Belt and suspenders.
AIMD for backpressure	Halve on 429, +1 on 200	Fixed pause + retry	AIMD converges to the agent's available capacity. Fixed strategies either over-throttle (wasting capacity) or under-throttle (repeated 429s).
Total timeout (queue + inflight)	30s covers both queue wait and inference time	Separate queue timeout + inference timeout	Single timeout is simpler for MFEs to reason about. "My request will complete or fail within 30s" — no surprises from invisible queue time.

Queue depth limit (32)	Bounded queue, reject at capacity	Unbounded queue	Unbounded queues silently accumulate latency. At 32 pending + 4 in-flight, the system is clearly overloaded — better to fail fast.
Cancel on unmount via AbortController	Full abort chain propagation	Let in-flight requests complete	Orphaned requests waste agent queue slots. AbortController is the browser-native mechanism — it sends TCP RST, freeing the agent's slot immediately.

14. Performance & Reliability

Performance

Metric	Value	Notes
Enqueue overhead	~0.1ms	Create QueueEntry + insert into priority lane
Dequeue (WFQ select)	~0.05ms	Compare 3 virtual finish times
Per-MFE fairness check	~0.02ms	Map lookup + integer comparison
Anti-starvation check	~0.01ms	Single counter comparison
Age promotion scan	~0.1ms	Iterate at most 32 entries
AbortController propagation	~0.01ms	Signal fires synchronously
Total scheduler overhead	< 0.3ms per request	Negligible vs inference latency (~50ms)
Memory overhead	~20KB	32-entry queue + 4 in-flight trackers + counters

Throughput under contention

Scenario	Effective throughput	Queue depth	P2 share
1 MFE, all P1	4 concurrent, no queueing	0	N/A
2 MFEs, mixed P0+P1	4 concurrent, fair 2+2 split	Low	WFQ-determined
2 MFEs, continuous P0 + bursty P2	4 concurrent, P2 gets 1/7	Moderate	14% + promotion
3 MFEs, all P1, bursty	4 concurrent, 1+1+1 + rotating 4th	High	N/A

Backpressure (429)	1-2 concurrent, AIMD recovery	High	Same WFQ ratios
--------------------	-------------------------------	------	-----------------

Reliability

Concern	Mitigation
Slot leak	Every code path (response, cancel, timeout, error) calls <code>release()</code> . Belt: a 60s sweep timer frees any slot held longer than the max timeout.
Starvation	WFQ weights + 8:1 hard guard + age-based promotion. Three independent mechanisms.
Queue overflow	Hard limit at 32 entries. Reject with <code>queue_full</code> error — MFE shows "system busy".
Agent overload	AIMD reduces concurrency on 429. Never sends more than <code>maxConcurrent</code> requests.
MFE crash	<code>AbortController</code> chain aborts all pending work. Slots freed. Other MFEs unaffected.
Timeout accumulation	Total timeout (queue + inflight) prevents silent latency. Age promotion escalates old requests.
Priority inversion	P0 preempts P2. Age promotion escalates stuck requests.

Reliability guarantees

- **No slot leak:** every path frees slots. Sweep timer as safety net.
- **No starvation:** WFQ + hard guard + age promotion — three independent defenses.
- **No MFE monopolization:** per-MFE quota enforced on every dispatch.
- **No silent queueing:** total timeout covers queue wait + inference time. MFE always gets a result or error within 30s (60s for background).
- **No 429 exposed:** scheduler absorbs backpressure via AIMD. MFEs see slower results, not HTTP errors.

Summary

Question	Answer
Queue structure	Three-level priority queue (critical/normal/background) with WFQ weights (4:2:1). 32-entry depth limit. Per-MFE sub-limits.
Scheduling policy	Weighted Fair Queueing — not strict priority. P0 gets 4/7 of dispatches, P2 gets 1/7. Anti-starvation hard guard (8:1). Age-based promotion (P2→P1 at 10s, P1→P0 at 20s).
Concurrency management	4-slot global limit. Per-MFE quota = $\text{floor}(4/N)$. Dynamic — adapts as MFEs mount/unmount. P0 can preempt P2 when all slots are busy.
Backpressure handling	AIMD on 429: halve concurrency, additive restore on 200. Never exposes 429 to MFEs. Queue absorbs excess locally. Reject at 32 entries (last resort).

Cancellation	AbortController chain: MFE → scheduler → fetch → TCP RST. Propagates to agent. Slots freed immediately. MFE unmount cancels all that MFE's work.
Starvation prevention	Three defenses: WFQ weights, 8:1 hard guard, age-based promotion. P2 is guaranteed service even under continuous P0 load.